МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

ФАКУЛЬТЕТ ПРИКЛАДНОЇ МАТЕМАТИКИ ТА ІНФОРМАТИКИ

ЗВІТ

До лабораторної роботи №4

Підготувала:

Студентка групи ПМА-43

Кравчук Марія

Мета роботи:

Ознайомитися з принципами та інструментами автоматизованого тестування веб-додатків. Навчитися писати модульні (Unit) тести для перевірки логіки програми, вимірювати рівень Code Coverage, проводити навантажувальне тестування та створювати скрипти для автоматизованого збору даних (Scraping).

Використані інструменти та технології:

- **Фреймворк веб-додатку:** Django (Python).
- **Модульне тестування:** вбудований модуль django.test
- **Аналіз покриття коду:** бібліотека coverage.
- **Навантажувальне тестування:** фреймворк Locust.
- **Автоматизація (скрапінг):** Selenium WebDriver

1. Модульне тестування та перевірка Code Coverage

Для перевірки працездатності основних маршрутів (URL) веб-додатку було створено набір модульних тестів у файлі tests.py. Тести використовують django.test.Client для імітації HTTP-запитів користувача до сервера. Було протестовано наступні сторінки:

- Головна сторінка (/)
- Сторінка карти (/map)
- Сторінка пошуку (/search та параметризований пошук /?q=Ukraine)
- Сторінка криптовалют (/cryptocurrencies)

За допомогою бібліотеки coverage було виміряно відсоток покриття коду тестами. Для файлу views.py (де міститься основна логіка відображення) вдалося досягти результату понад 30%, що задовольняє вимоги завдання. Щоб уникнути конфліктів зі статичними файлами під час тестування, було застосовано перевизначення сховища (FileSystemStorage).

```
(venv) PS C:\WhereCrypto-main> python manage.py test country_search
Found 5 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
..C:\WhereCrypto-main\venv\lib\site-packages\pyproj\crs\crs.py:141: FutureWarning:
'+init=<authority>:<code>' syntax is deprecated. '<authority>:<code>' is the prefer
red initialization method. When making the change, be mindful of axis order changes
: https://pyproj4.github.io/pyproj/stable/gotchas.html#axis-order-changes-in-proj-6
  in_crs_string = _prepare_from_proj_string(in_crs_string)
...
----------------------------------------------------------------------
Ran 5 tests in 0.722s

OK
```

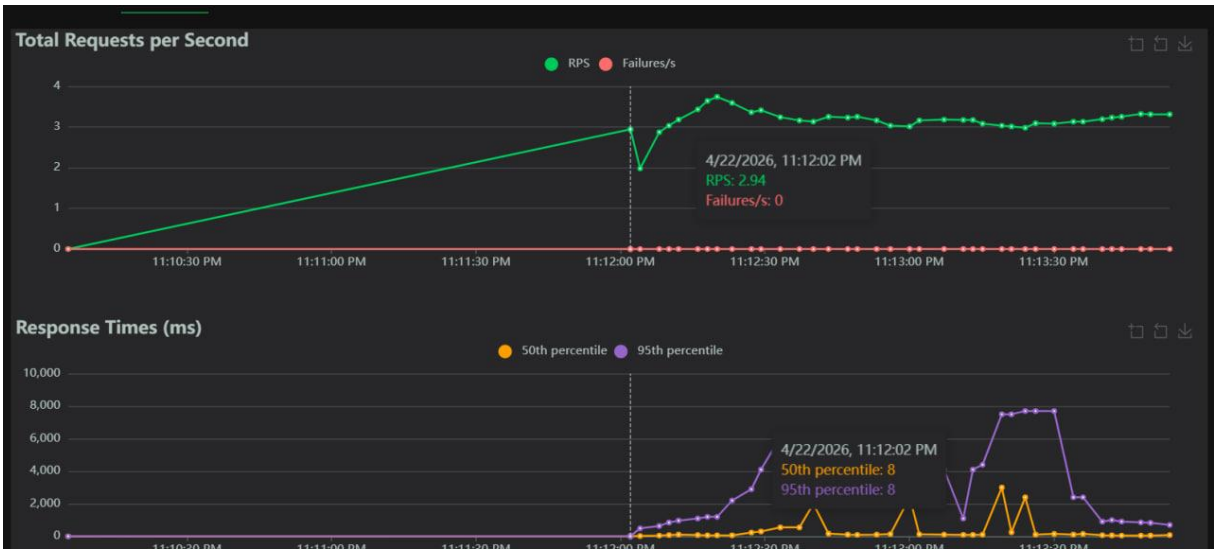
```
(venv) PS C:\WhereCrypto-main> coverage report
Name                                    Stmts   Miss  Cover
-----------------------------------------------------------
country_search\__init__.py                  0      0   100%
country_search\admin.py                     1      0   100%
country_search\apps.py                      4      0   100%
country_search\migrations\__init__.py       0      0   100%
country_search\models.py                    1      0   100%
country_search\tests.py                    25      0   100%
country_search\urls.py                      4      0   100%
country_search\views.py                   121     72    40%
-----------------------------------------------------------
TOTAL                                     156     72    54%
(venv) PS C:\WhereCrypto-main>
```

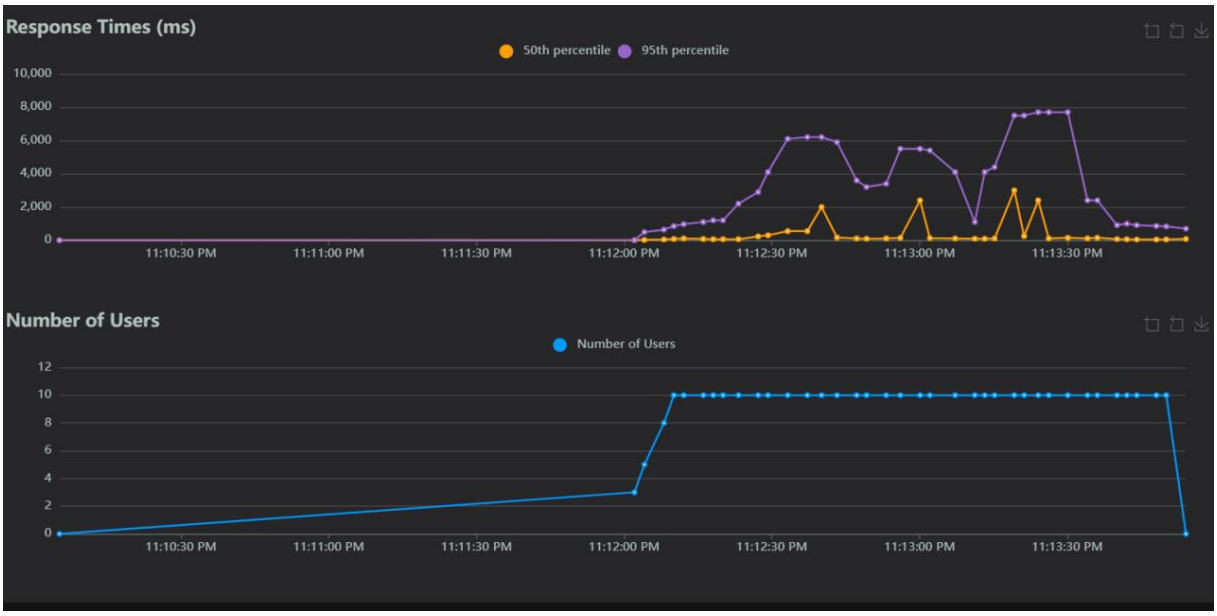
2. Навантажувальне тестування за допомогою Locust

Для перевірки стійкості додатка до навантажень було створено файл locustfile.py. У цьому файлі описано поведінку користувача , який виконує складний перелік дій:

1. Заходить на головну сторінку.
2. Виконує пошуковий запит із передачею параметра (наприклад, шукає країну "Ukraine").
3. Переходить на сторінку з картою.

Тестування проводилося локально з імітацією одночасного доступу 10 користувачів (швидкість появи — 1 користувач на секунду). Графіки продемонстрували стабільну роботу сервера без критичних затримок та помилок .





Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/	117	0	47	190	500	67.33	2	515	4596	1.5	0
GET	/map	134	0	1100	6100	7700	2220.9	428	8180	594171	1.4	0
GET	/search	117	0	68	250	550	82.95	1	597	4588	1.5	0
	Aggregated	368	0	110	4600	7500	856.48	1	8180	219275.66	4.4	0

3.Автоматизований збір даних (Scraping) за допомогою Selenium

Для виконання завдання з автоматизації було написано Python-скрипт scraper.py із використанням Selenium WebDriver. Скрипт виконує наступні дії:

- Автоматично ініціалізує та відкриває браузер Google Chrome (за допомогою webdriver_manager).
- Переходить на локальну адресу розгорнутого веб-додатку.
- Очікує завантаження сторінки.
- Зчитує та виводить у консоль заголовок сторінки.
- Завершує сесію та закриває браузер.

```
(venv) PS C:\WhereCrypto-main> python scraper.py
Запуск автоматичного скрапера...
Переходимо за адресою: http://127.0.0.1:8000/

========================================
[SUCCESS] Скрапінг успішний!
Заголовок сайту: 'WhereCrypto'
========================================

Браузер закрито.
```

Висновок:Під час виконання лабораторної роботи було успішно застосовано комплексний підхід до перевірки якості веб-додатку на базі Django. Написані модульні тести підтвердили коректність маршрутизації та обробки запитів, забезпечивши необхідний рівень покриття коду. Навантажувальне тестування за допомогою Locust довело здатність сервера обробляти паралельні запити від кількох користувачів без збоїв. Використання Selenium дозволило автоматизувати процес взаємодії з веб-інтерфейсом та збору даних.